CST2550 Coursework:

Train Tracking & Arrival Prediction

GROUP VITALITY

Casey Tyler-Woods

Ishaan Hari

Rishon Ramdany

Naga Arun Mahesh

TABLE OF CONTENTS:

TABLE OF CONTENTS: **1**
INTRODUCTION **1**
Project Overview 1
Report Structure 1
DESIGN **1**
Testing **6**
Conclusion **7**
References **8**

INTRODUCTION

Project Overview

As the name suggests, the focus of our project is to allow users to view live positions of trains and their travel routes. Users can create an account to access additional features such as saving favourite stations and viewing live journeys on an interactive map etc.. The system integrates real-time data from the Transport for London(TFL) API to provide accurate and live train information and railway network statuses.

Report Structure

This report is segregated into four main sections. The Design section explains the data structures and algorithms used, along with justification for design decisions. The Testing section outlines the testing approach. The Conclusion summarises the project, reflects on limitations, and suggests possible improvements in our workflow. Finally, the References section lists all sources used in Harvard format.

DESIGN

Design Methodology(MoSCoW)

In the designing phase of our project we chose to dissect the problem requirements into the MOSCOW develop method. The goal of this method is to sort potential features of the system into groups of varying importance, which helped to streamline where we focused our efforts in the early stages and middle stages of development. We took potential features that we had come up with and filtered them into four categories based on their importance.

Program requirements following the MOSCOW development method:
Mo - Must have (Requirements necessary to the completion of the process)
S - Should have (Requirements that are important, but not essential)
Co - Could have (Nice to have requirements that have a small impact if left out)
W - Won't have (Requirements that are not important and will be left out)

Must have:
-Train station database (station position, number of platforms, platforms size)
-Train database (number, carriages, company, average speed)
-User account creation - Users database (username, password, age, email)
-Train travel time calculation function
-Train arrival at station calculation function
-Train position prediction function
-Delays detection and alert
-Route viewing

Should have:
-Track disruptions or cancellations
-Station arrivals and departures
-Device friendly ui
-Basic user settings (favourite/commonly searched stations, recent searches)

Could have:
-Alerts signaling train approaching or delays.
-Journey planner
-Train occupancy predictions

Won't have:
-Ticket booking service
-Machine learning prediction model
-Multi country rail networks

In our must have section we decided on three main data structures that our system would require being for the USER, TRAIN, & STATION .

User: As for these data structures, the user is required for setting and storing the key information of the user's account, which is a standard feature of most websites and will be necessary for utilising some lesser important features as can be seen in the should have and could have sections. The four main attributes we decided on were username, password, age and email. Username is a key identifier for an account, password and email are both to ensure security and age to track usage demographic.

Data structures to represent trains and stations are much more specific to our system, with data from both being required for the basic functions that we want our system to be capable of.

Trains: The attributes of trains that we saw fit were the train's number identifier, number of carriages, the company operating it and the speed the service is operating at. The number identifier is stored a string and helps us to track the exact train and to be able to locate it in our database, number of carriages is stored as an int and can help in predicting space availability on the train, the company operating can be useful information for the user to view and the speed of the service is both necessary for creating arrival time predictions as well as informing users whether a train is considered a 'fast train' or 'slow train'.

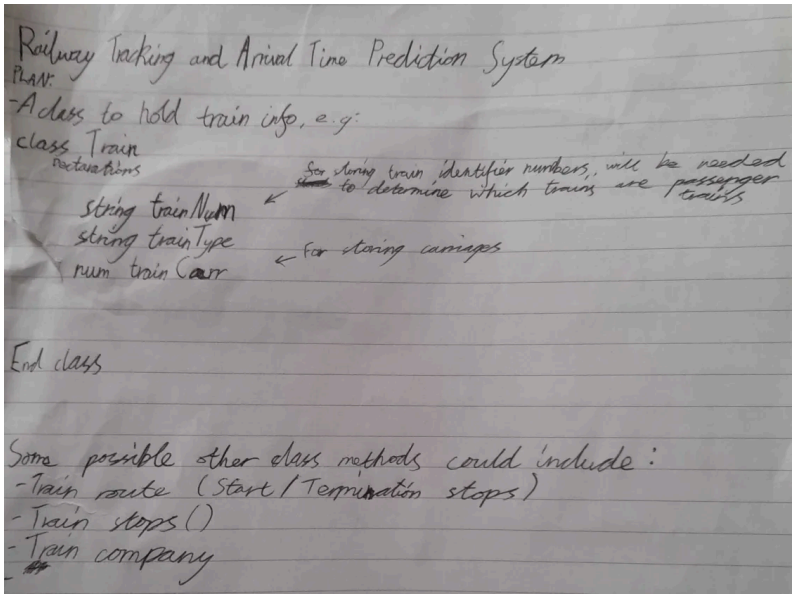
Station: As for the station data structure, the attributes we initially went with were the station's position, the number of platforms and the platform size. The position is to store the geographical coordinates of the station which we did by splitting it into two integers storing the longitude and latitude values respectively of the station to use in calculating train travel times between stations.

The number of platforms is relevant information to display to the user by indicating the arrival platform of the train. The platform sizes are further relevant information to display to the user, as certain stations have smaller platforms than average train lengths.

As part of the development process, the Train model was extended to include an average speed attribute, which was outlined in the project brief as a required property.
Additionally, a password hashing implementation was added to the User model using the BCrypt library, ensuring that user passwords are stored securely rather than as plain text.
The stations database table was also created and populated with the coordinates of all Elizabeth Line stations, which provided the geographical data needed for the mapping features.

And as part of the could have requirements, an AlertService class was developed to handle two types of alerts - one to detect train delays and the other for flagging trains arriving within 2 minutes. Also a Live Map page was added to the website, which displays an interactive map of the Elizabeth Line using Leafjets.js with live train positions pulled from the TFL APi.
Users can search by station name or by the Trains ID to locate the specific trains.

Below are some initial ideas of the early stages of the plan for the data structures done on paper:


Railway Tracking and Arrival Time Prediction System
PLAN:
-A class to hold train info, e.g:
class Train
Declarations
For storing train identifier numbers, will be needed to determine which trains are passenger trains
string trainNum
string trainType
num trainCarr
For storing carriages
End class
Some possible other class methods could include:
-Train route (Start / Termination stops)
-Train stops()
-Train company

As for the points classed as won't have, the idea of ticket booking as a function came up multiple times in early design stages. We decided that it is not a feature that fits the intended scope of our project and therefore decided against its inclusion. A machine learning prediction model and multi-country rail networks are both potential features that are far out of the scope of the project and were not included.

After creating our initial plan, we made a new streamlined plan for development. We broke down points into classifications of objectives, key features and technologies we would use. Once these points were clarified, we created a timeline to lay out the order our development process would take.

Project Plan

Project Title: Where's the Train?

Overview:
This project is a web-based train tracking application focused on the Elizabeth Line. It allows users to check train departures, view disruptions, and manage favourite stations.

The system was built step by step, starting with the backend structure, then adding a database, followed by the frontend, and finally integrating live data using the TfL API.

Objectives:
- Build a working ASP.NET web application
- Create a backend structure to support features
- Design and implement a database for user data
- Develop a user interface for interaction
- Integrate the TfL API for live train and disruption data

Key Features:
- Train departures board
- Live service alerts and disruptions
- User account system
- Favourite stations feature
- Alert notification system

Technologies Used:
- ASP.NET Core Razor Pages
- C#
- SQL (database for users and favourites)
- JavaScript (for live updates)
- CSS & HTML (for frontend style)
- Bootstrap (styling)
- TfL API

Development Approach:
The project began with setting up the backend structure using ASP.NET. A database was then designed to store user information and favourite stations. After this, the frontend pages were developed, including the account page and departures page. Once the core system was working, the TfL API was integrated to provide live train data and disruption information. Additional features such as alerts, notifications, and UI improvements were added in later stages. The final phase focused on testing, fixing bugs, and improving consistency across the application.

Timeline

Project Timeline

Week 1:
- Set up ASP.NET project
- Build backend structure
- Create basic models and pages

Week 2:
- Design and implement SQL database
- Store user information and favourites
- Test database connections

Week 3:
- Develop frontend pages (Account, Departures)
- Display data from backend
- Improve layout and navigation

Week 4:
- Integrate TfL API
- Fetch live train and service data
- Display departures in real time

Week 5:
- Implement alerts system
- Handle disruption data from API
- Add alert badge and notifications

Week 6:
- Improve UI styling across all pages
- Fix bugs and inconsistencies
- Final testing and preparation for submission

Testing

Our testing approach was to ensure minimal usability errors for users. We focused on preventing errors from commonly made user mistakes, things such as invalid or missing entries when filling in fields. We also ensured that we tested areas which are more prone to human error and tried to break our website logic with unit testing and various types of input values(valid, boundary, invalid).

Test Cases:
User test case:

Function:	Scenario:	Steps:	Expected Result:	Actual Result:
Register	Valid registration details entered	Navigate to register page. Enter a valid username, password, email, age and phone.	Account is successfully created and user is logged in.	Account is successfully created. User is logged in and returned to the home page.
Register	Invalid registration details entered	Navigate to register page. Enter an invalid input type into one of the fields. (e.g. entering a string into phone number field or a non email address into email field)	Will result in an error and user will be kept on registration page to retry.	Account is created with invalid inputs, which may cause negative effects when attempting use of other features.
Register	Empty fields	Navigate to register page. Attempt to register with one or more fields left empty.	Will result in an error and user will be kept on the registration page to retry.	Account is created with missing data.
Login	Valid login details entered	Navigate to login page. Enter valid inputs into the username and password fields.	Account is logged into successfully and user is returned to the home page.	User is taken to the home page and successfully logged in.
Login	Invalid login details entered	Navigate to login page. Enter an invalid input for either of the fields.	Error message displayed. User is kept on login page to retry.	‘Invalid login’ error message displayed and user is kept on the login page.
Login	Empty fields	Navigate to login page. Attempt to login while leaving either of the fields empty.	Error message displayed. User is kept on login page to retry.	If password is empty then error message is displayed and user is kept on login page. If username (or both) is empty an internal server error occurs.

Functions test case:

Function:	Scenario:	Steps:	Expected Result:	Actual Result:
Route search	Valid starting station and destination search	Navigate to track trains page. Enter valid stations for both fields.	Displays a list of available trains on selected route.	Displayed list of available trains on the selected route.
Route search	Invalid starting station and destination search	Navigate to track trains page. Enter invalid options for either or both fields.	Error message displayed. User kept on track trains page.	Error message is displayed and user is kept on page to try entering again.
Route search	Empty starting station and destination search	Navigate to track trains page. Leave either or both fields empty.	Error message displayed. User kept on track trains page.	Error message is displayed and user is kept on the page to retry.
Departures and arrivals search	Valid station search	Navigate to departures and arrival page. Enter a valid station into the field.	Displays info on upcoming departures and arrivals for the entered station.	The upcoming departures and arrivals are displayed.
Departures and arrivals search	Invalid station search	Navigate to departures and arrival page. Enter an invalid input.	Error message displayed. User kept on page.	Error message is displayed and the user is kept on the page to retry.
Alerts page	Page loaded	Navigates to alerts page	Live disruptions and approaching trains displayed.	Disruptions and approaching trains loaded from TFL.
LIved Map search	Invalid search entered	Navigate to live map, Enter an invalid station or train ID	No results shown, map stays unchanged.	No results displayed.
Live Map search	Valid search entered	Navigate to a live map . Enter a valid station name.	Map zooms to station, trains filtered and displayed.	Map zoomed to the station and matching trains were shown.

Conclusion

Overall, we developed a functional website that allows users to find information on train routes and services passing through specific stations, while also incorporating a range of features to enhance the user experience. We ensured the layout adapts to different screen sizes, making the site more accessible, which is particularly important for a service that users may need to check quickly on their phones while on the move.

The development of our project was hindered by a number of factors. The absence of a fifth team member created challenges, particularly during the early stages of development, leading to initial uncertainty as the group anticipated an additional member who ultimately did not join. Not having a fifth member affected how much we were able to accomplish, as members had to extend their work beyond their predefined roles to compensate.

While we encountered some challenges in maintaining consistent communication, particularly due to less frequent in-person meetings and occasional gaps in our WhatsApp discussions, the team demonstrated strong adaptability and commitment throughout the project. Despite these obstacles, we made a conscious effort to stay aligned, collaboratively resolving any disconnects in our code. Furthermore, the group worked hard and efficiently, eventually overcoming the challenges and completing the project to the best of our abilities.

Our professionalism and group dynamic was highly valued amongst the team members. In the future, adopting a more systematic approach to scheduling and task allocation would improve overall productivity and time efficiency. Additionally, defining responsibilities and setting timelines for each stage of the project would help ensure smoother development, stronger coordination, and more effective task completion.

References

- *Transport for London (TfL), 2026. Unified API Documentation. Available at: https://api.tfl.gov.uk (Accessed: April 2026)*
- *Leaflet.js, 2026. Leaflet — an open-source JavaScript library for interactive maps. Available at: https://leafletjs.com (Accessed: April 2026)*
- *OpenStreetMap, 2026. OpenStreetMap map tiles. Available at: https://www.openstreetmap.org (Accessed: April 2026)*
- *BCrypt.Net, 2026. BCrypt.Net-Next NuGet package. Available at: https://github.com/BcryptNet/bcrypt.net (Accessed: April 2026)*